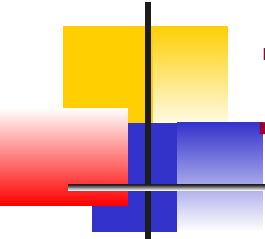


第6章

输入输出及中断技术

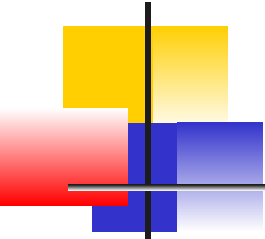




主要内容

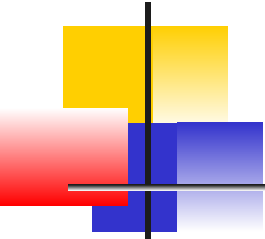
- 输入输出系统的基本概念
- I/O接口和端口
- 端口的编址方式
- 简单接口芯片及其应用
- 基本输入输出方法

§6.1 输入输出系统



了解和掌握：

- I/O系统的概念和特点
- 接口的基本功能
- 端口的概念
- 端口的编址方式
- I/O地址译码



一、输入输出系统的组成及特点

- **组成：**

- I/O设备， I/O接口， I/O软件

- **特点：**

- 复杂性

- 实时性

- 异步性

- 与设备无关性

二、I/O接口

I/O接口要解决的问题

- 速度匹配(Buffer) 数据的缓冲与暂存
- 信号的驱动能力(电平转换器、驱动器) 信号驱动
- 信号形式和电平的匹配(A/D、D/A) 信号类型转换
- 信息格式(字节流、块、数据包、帧)
- 时序匹配(定时关系)
- 总线隔离(三态门)



接口的功能

- 数据的缓冲与暂存
- 信号电平与类型的转换
- 增加信号的驱动能力
- 对外设进行监测、控制与管理，中断处理

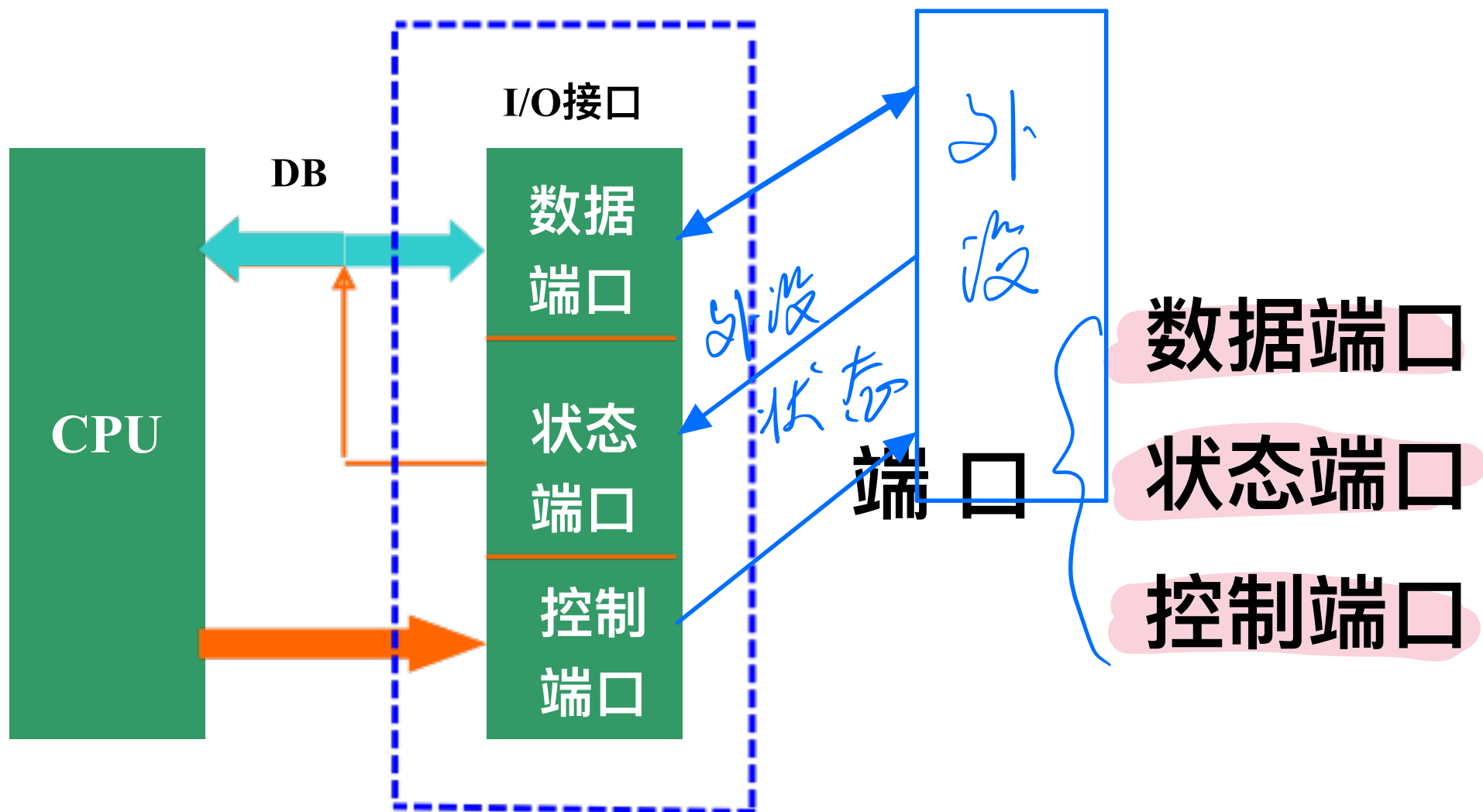
I/O接口和端口

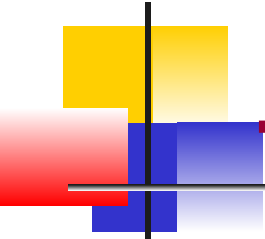
I/O接口：

- 将外设连接到总线上的一组逻辑电路的总称。
- 实现外设与主机之间的信息交换。

I/O端口：

- 接口中的**寄存器**





三、I/O端口的编址方式

- 8086/8088的寻址能力：

- 内存：

- 1MB

- 端口：

- 64KB

- 编址方式：

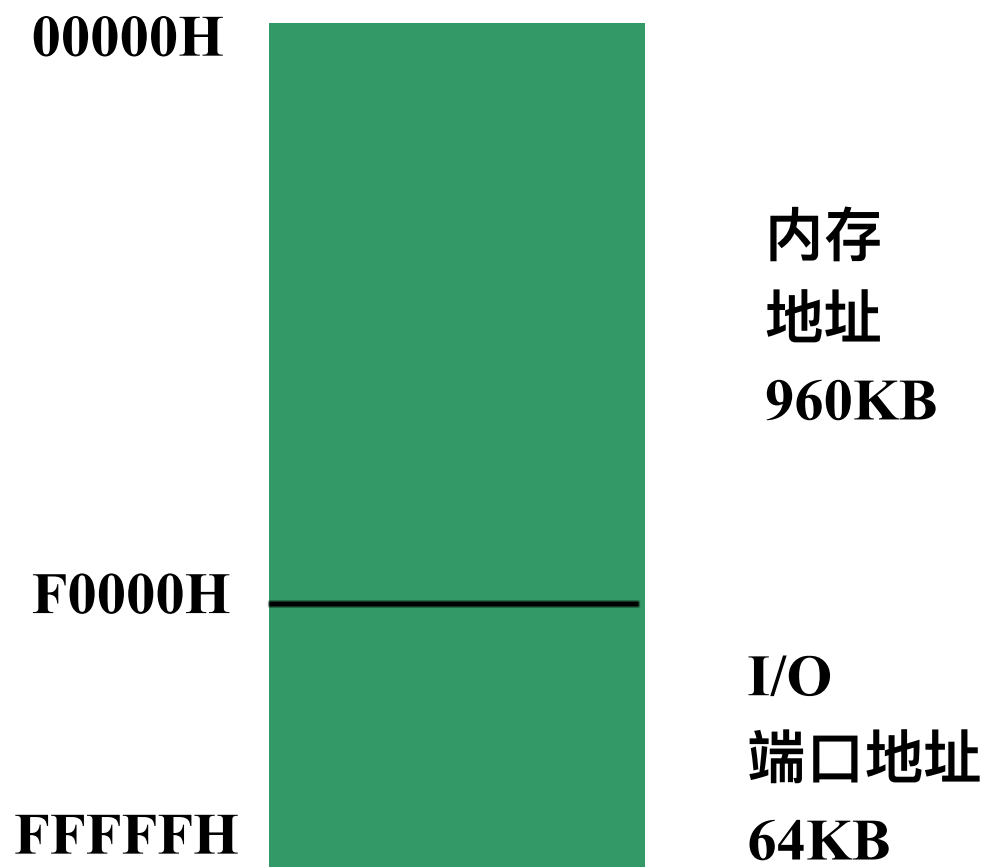
- 与内存统一编址

- 独立编址

端口与内存的统一编址

特点:

- 指令及控制信号统一
- 内存地址资源减少



端口的独立编址

特点:

- 内存地址资源充分利用
- 能够应用于端口的指令较少

00000H

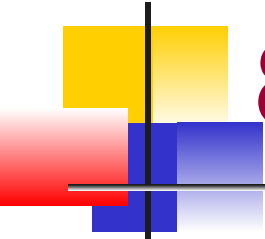
FFFFFFH

0000H

FFFFFH

内存
地址

I/O
地址



8088/8086的I/O端口独立编址

- 采用I/O独立编址方式(但地址线与存储器共用)
- 地址线上的地址信号用IO/ $\overline{M}$ 来区分； $\overline{IOR}$ 、 $\overline{IOW}$ 控制读写
- I/O操作只使用20根地址线中的16根：A15~A0
- 可寻址的I/O端口数为64K(65536)个
- I/O地址范围为0~FFFFH
- IBM PC只使用了1024个I/O地址(0~3FFH)



四、I/O端口地址的译码

目的:

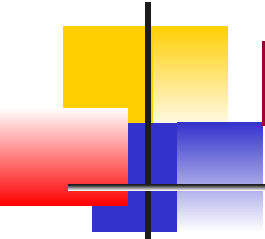
- 确定端口的地址

参加译码的信号:

- IOR, IOW, 高位地址信号
- OUT指令将使总线的IOW信号有效
- IN指令将使总线的IOR信号有效

$$Io/\bar{M} = 1 \quad \overline{RD} = 0 \Rightarrow \overline{IOR} = 0$$

$$\overline{WR} = 0 \Rightarrow \overline{IOW} = 0$$



I/O译码的地址信号

- 当接口只有一个端口时，16位地址线一般应全部参与译码，译码输出直接选择该端口；

----全部译码方式

- 当接口具有多个端口时，则16位地址线的高位参与译码（决定接口的基地址），而低位则用于确定要访问哪一个端口

----高位译码选接口，低位决定片内端口地址

每个端口地址 = 片选地址(高位) + 片内地址

I/O地址译码例

某外设接口有4个端口，地址为2F0H——2F3H，由A15～A2译码得到，而A1、A0用来区分接口中的4个端口。试画该接口与系统的连接图。

- 地址范围：

• ^{A11} × × × × ^{A9} 0 0 1 0 1 1 1 1 0 0 0 0

• × × × × 0 0 1 0 1 1 1 1 0 0 1 1

任意状态

片内地址

图中不接入

I/O地址译码例

地址为2F0H——2F3H

译码电路图：

$\overline{IOR}$
 $\overline{IOW}$

CPU地址
2若不同
时有效
(不同为0)

A11

A10

A8

A3

A2

A9

A7

A4

≥ 1

⋮

&

A1

A0

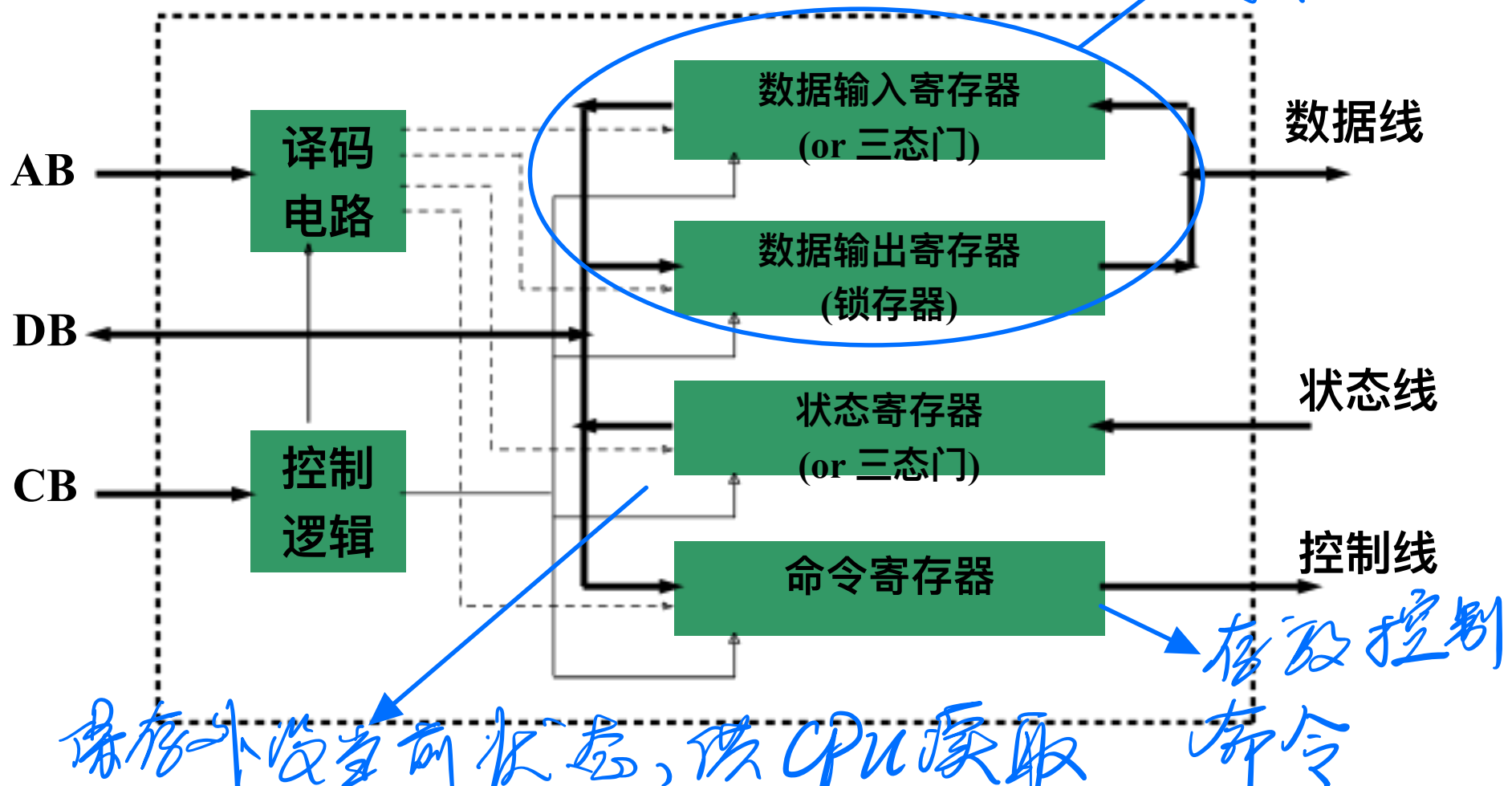
接口芯片

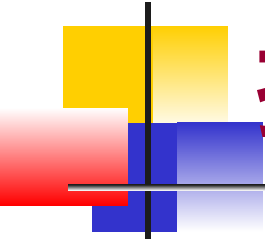
$\overline{CE}$

§6.2 简单接口电路

- 接口电路的分类及特点;
- 两类简单接口芯片的应用

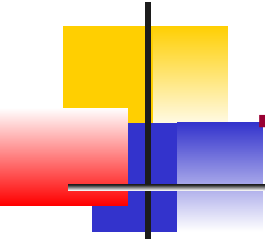
一、接口的基本构成





接口的基本构成

- 数据输入/输出寄存器(端口)
 - 暂存输入/输出的数据
- 命令寄存器
 - 存放控制命令
 - 设定接口功能、工作参数和工作方式。
- 状态寄存器
 - 保存外设当前状态，以供CPU读取。



二、接口的类型及特点

- 按传输信息的**方向**分类：

- 输入接口
- 输出接口

- 按传输信息的**类型**分类：

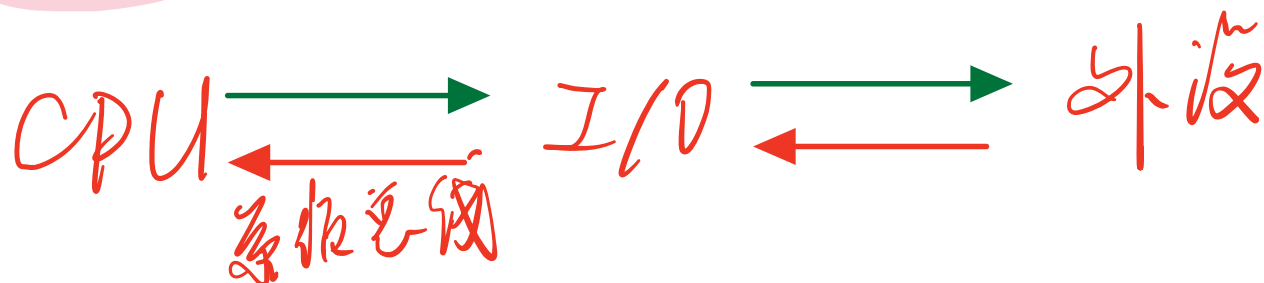
- 数字接口
- 模拟接口

- 按传输信息的**方式**分类：

- 并行接口
- 串行接口

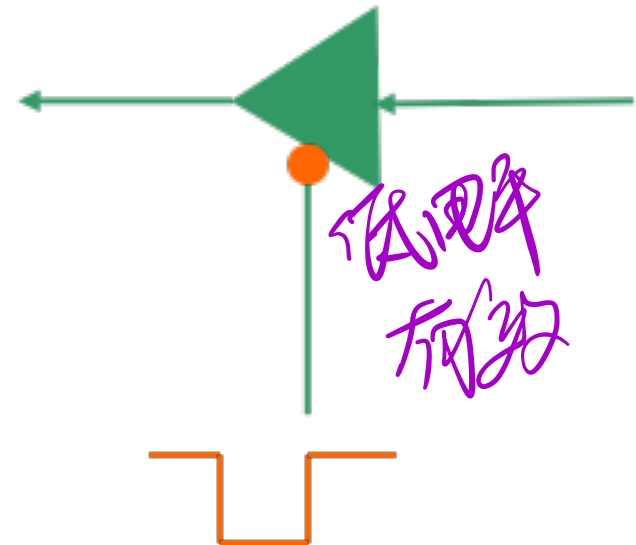
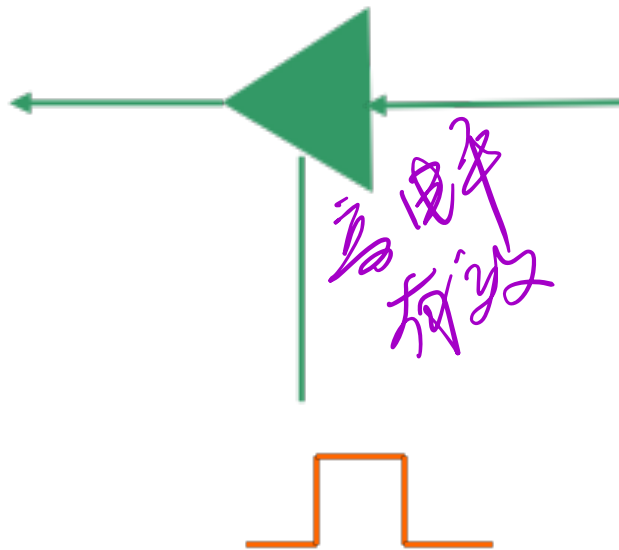
接口特点!!!

- 输入接口:
 - 要求对数据具有控制能力
 - 常用三态门实现
- 输出接口:
 - 要求对数据具有锁存能力
 - 常用锁存器实现



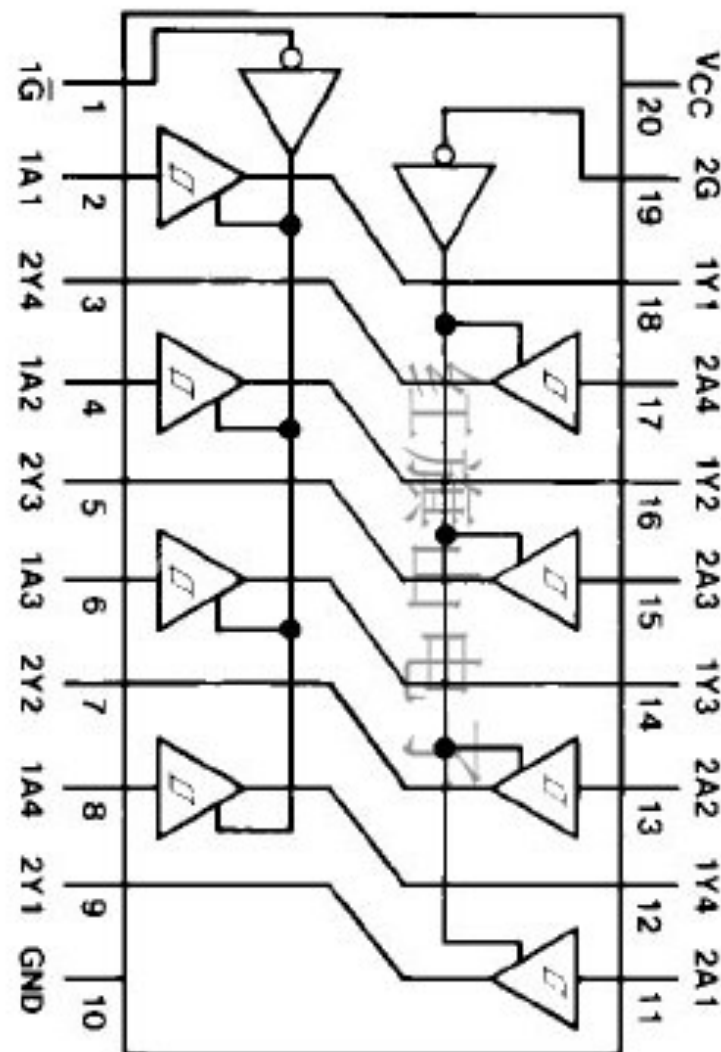
三、三态门接口

- 高电平、低电平、高阻态



74LS244

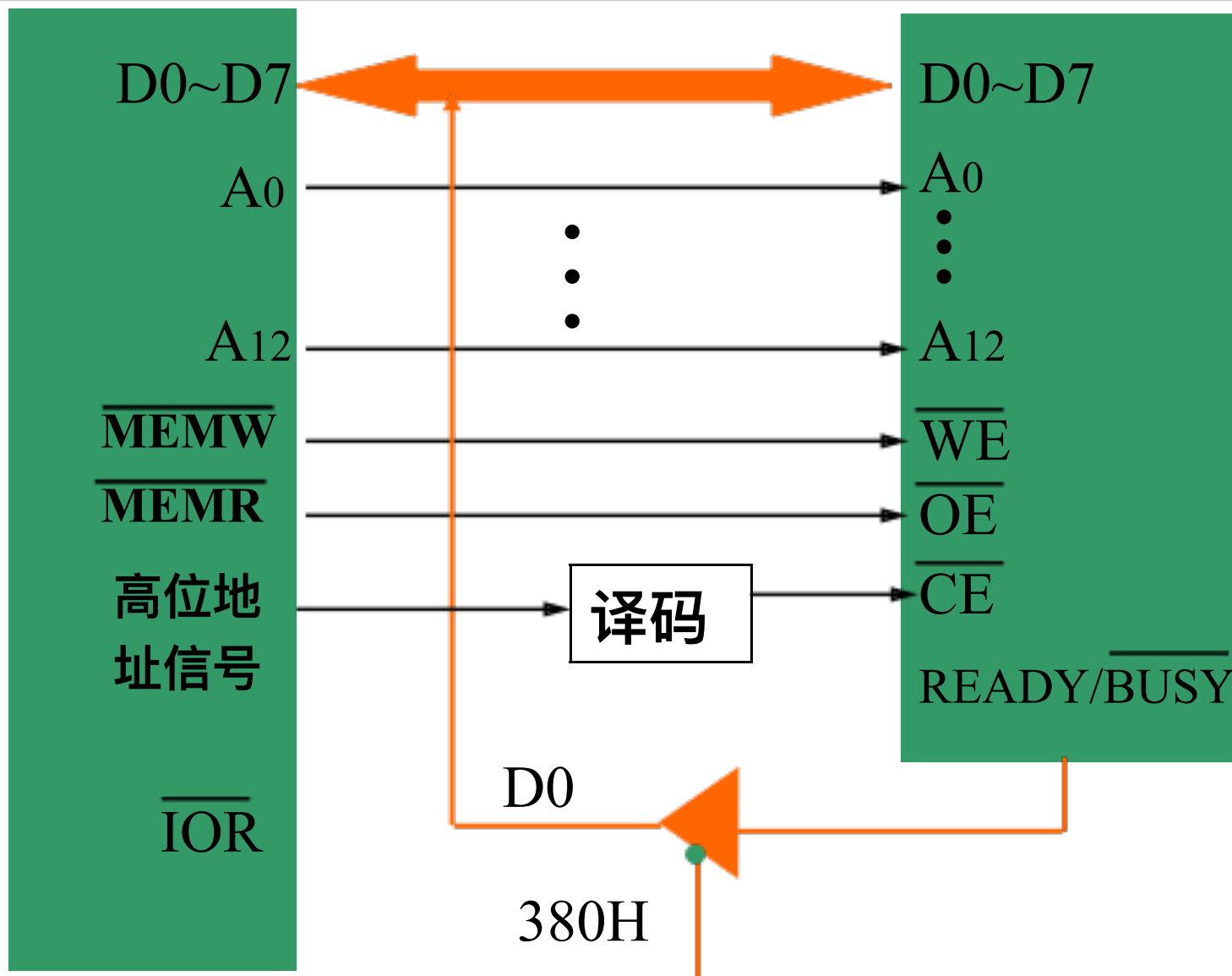
- 含8个三态门的集成电路芯片
- 在外设具有数据保持能力时用来输入接口
- 74LS244应用例——教材p238



P238图

三态门接口应用例

- 利用三态门作为输入接口（接口地址380H）接到地址范围为70000H---71FFFH的EEPROM芯片的READY/BUSY端，当三态门输出高电平时，可向98C64A写入一个字节数据，输出低电平时则不能写入。



四、锁存器接口

通常由D触发器构成；

特点：

- 具有对数据的锁存能力；

- 一般不具备对数据的控制能力

- 74LS273

8D触发器，不具备数据的控制能力

只能做输入

出接口

P239图

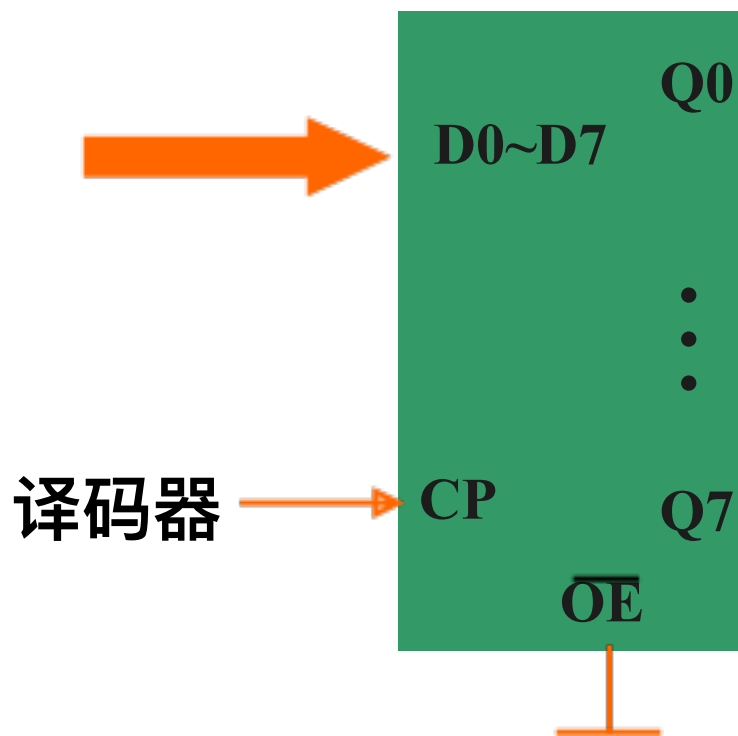
- 74LS374

含三态的8D触发器，具有对数据的控制能力

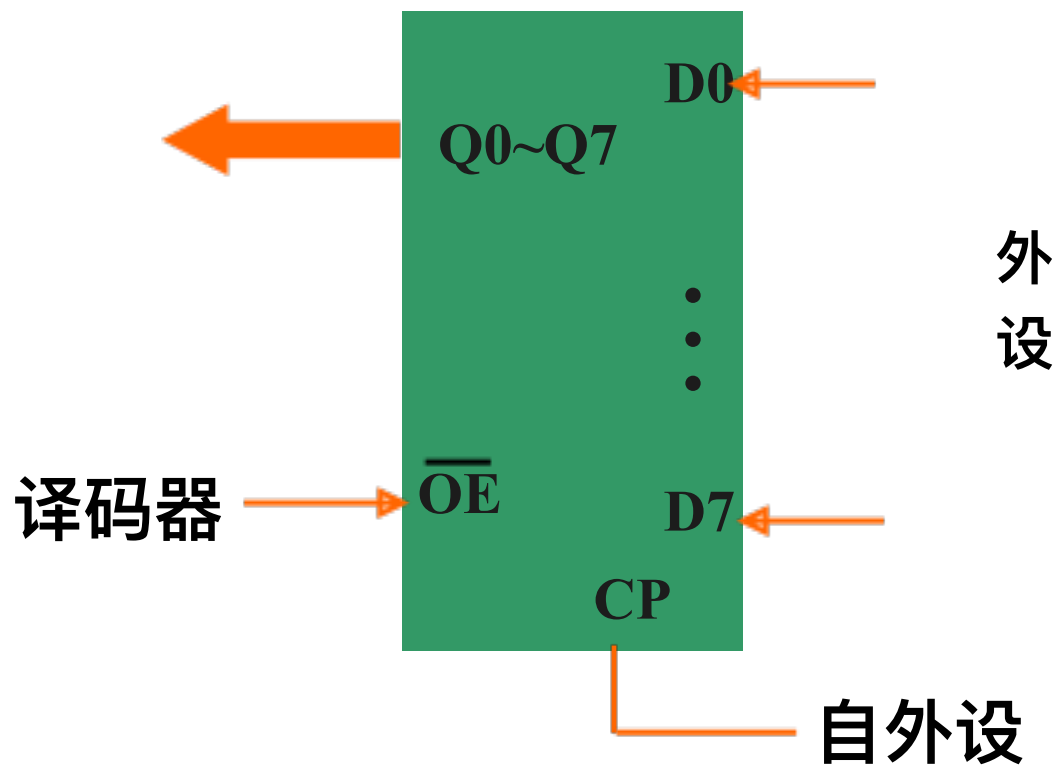
既可做输入，也可做输出

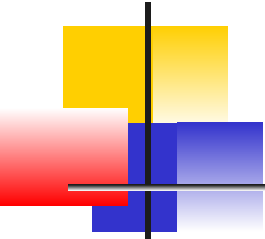
锁存器芯片74LS374

做输出口:



做输入口:





I/O接口综合应用例

- 根据开关状态在7段数码管上显示数字或符号
- 设输出接口的地址为F0H
- 设输入接口地址为F1H
- 当开关的状态分别为0000~1111时，在7段数码管上对应显示'0'~'F'

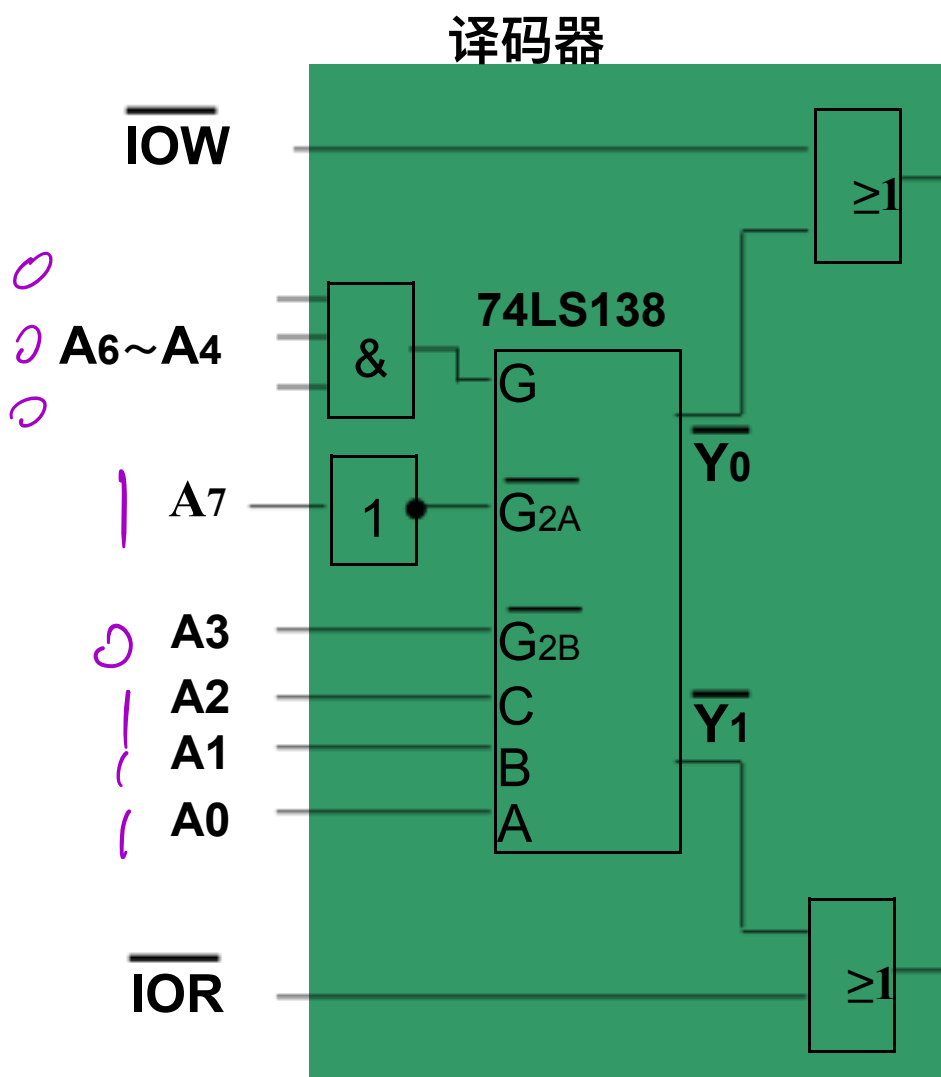
7段数码管图见教材p241

符号	形状	7段码 .gfedcba	符号	形状	7段码 .gfedcba
'0'		00111111	'8'		01111111
'1'		00000110	'9'		01100111
'2'		01011011	'A'		01110111
'3'		01001111	'B'		01111100
'4'		01100110	'C'		00111001
'5'		01101101	'D'		01011110
'6'		01111101	'E'		01111001
'7'		00000111	'F'		01110001

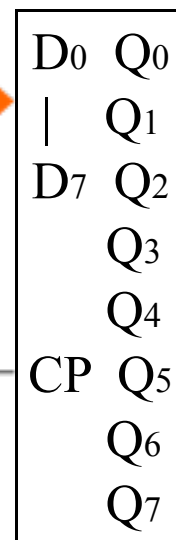
F0H = 1111 0000
F1H = 1111 0001

↗ A0

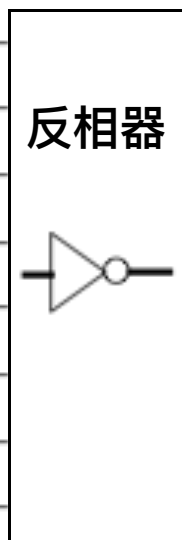
D0~D7



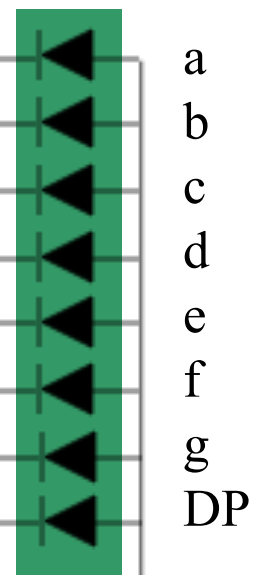
74LS273



7406

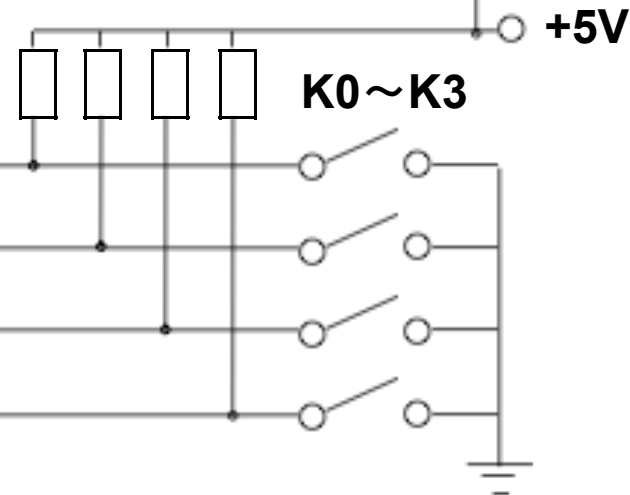
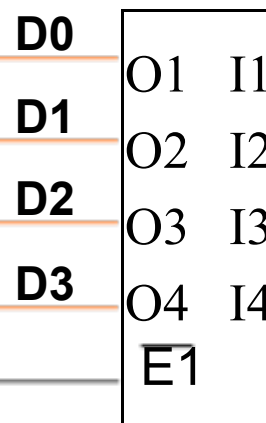


Rx8



+5V

74LS244



I/O接口综合应用例 —— 程序段

.....
Seg7 DB 3FH,06H,
5BH,4FH,66H,6DH,
7DH,07H,7FH,67H,77H,
7CH,39H,5EH,79H,71H
.....

LEA BX, Seg7
MOV AH, 0

GO: IN AL, 0F1H
AND AL, 0FH 读开关状态

MOV SI, AX 开关状态的
编码

MOV AL, [BX+SI]

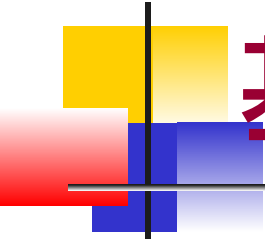
OUT 0F0H, AL

JMP GO

↓
开关状态
对应的七段码
值的编码



§6.5 基本输入/输出方法



基本输入/输出方法

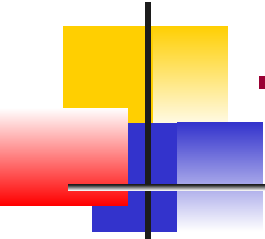
无条件传送

查询式传送

中断方式传送

直接存储器存取(DMA)

} 程序控制方式

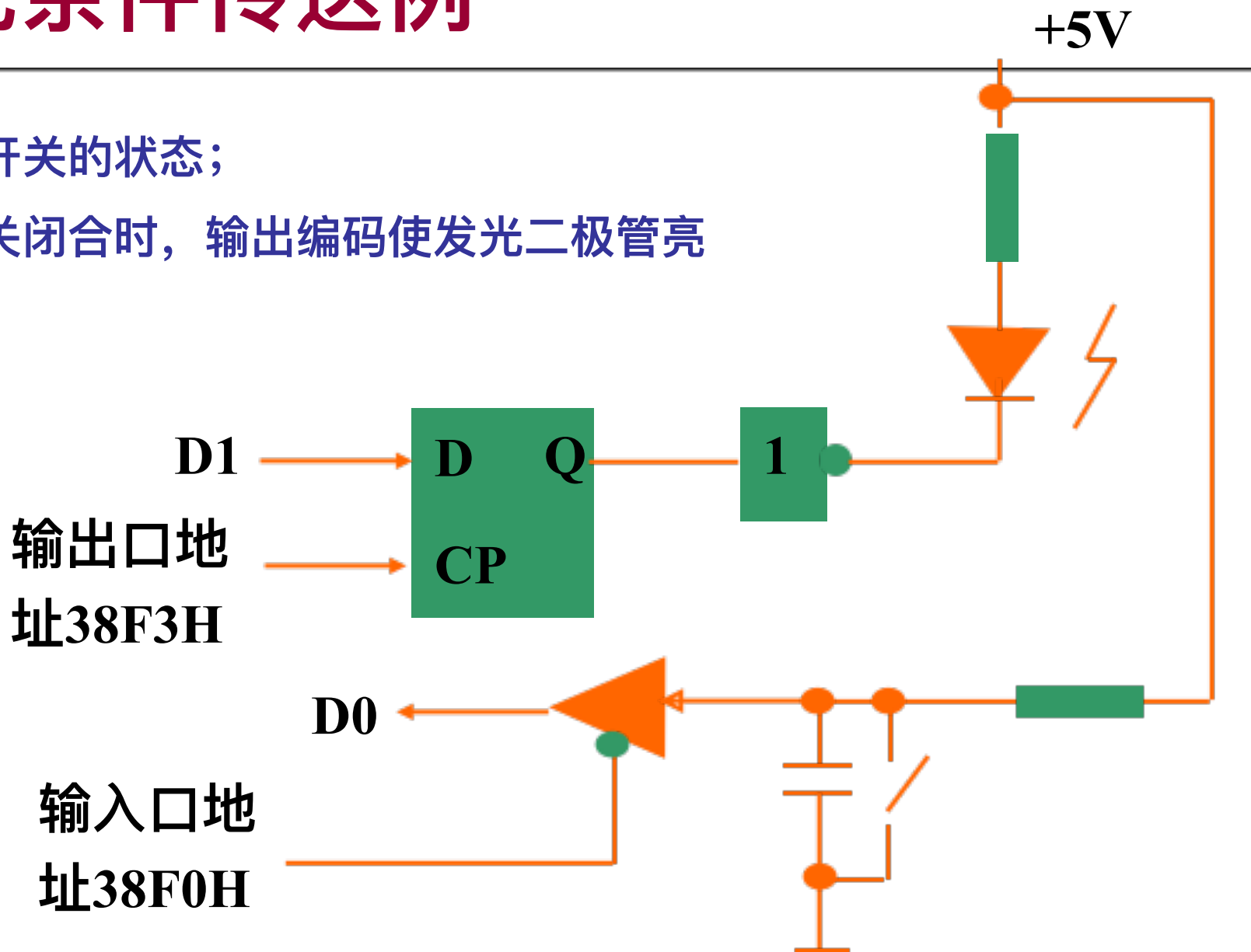


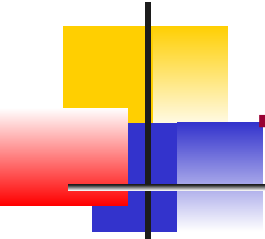
一、无条件传送

- 要求外设总是处于准备好状态
- 优点：
 - 软件及接口硬件简单
- 缺点：
 - 只适用于简单外设，适应范围较窄

无条件传送例

- 读取开关的状态;
- 当开关闭合时, 输出编码使发光二极管亮

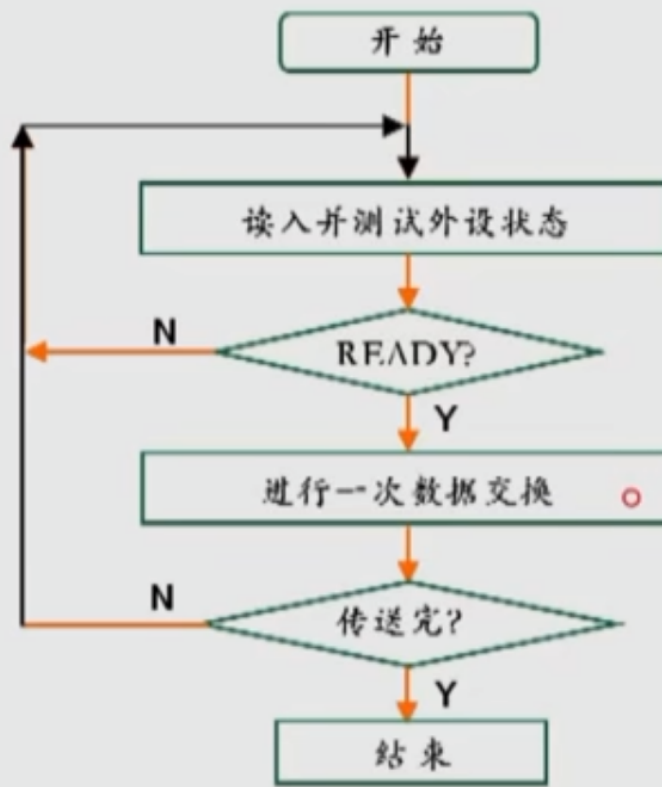




二、查询工作方式

- 仅当条件满足时才能进行数据传送；
- 每满足一次条件只能进行一次数据传送。
- 适用场合：
 - 外设并不总是准备好
 - 对传送速率和效率要求不高
- 工作条件：
 - 外设应提供设备状态信息
 - 接口应具备状态端口（寄存器）

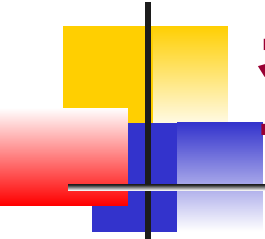
查询工作方式流程图



每满足一次
条件只能进
行一次数据
传送

优点：软硬件简单

缺点：CPU 效率低，数据传送的实时性差，速度较慢

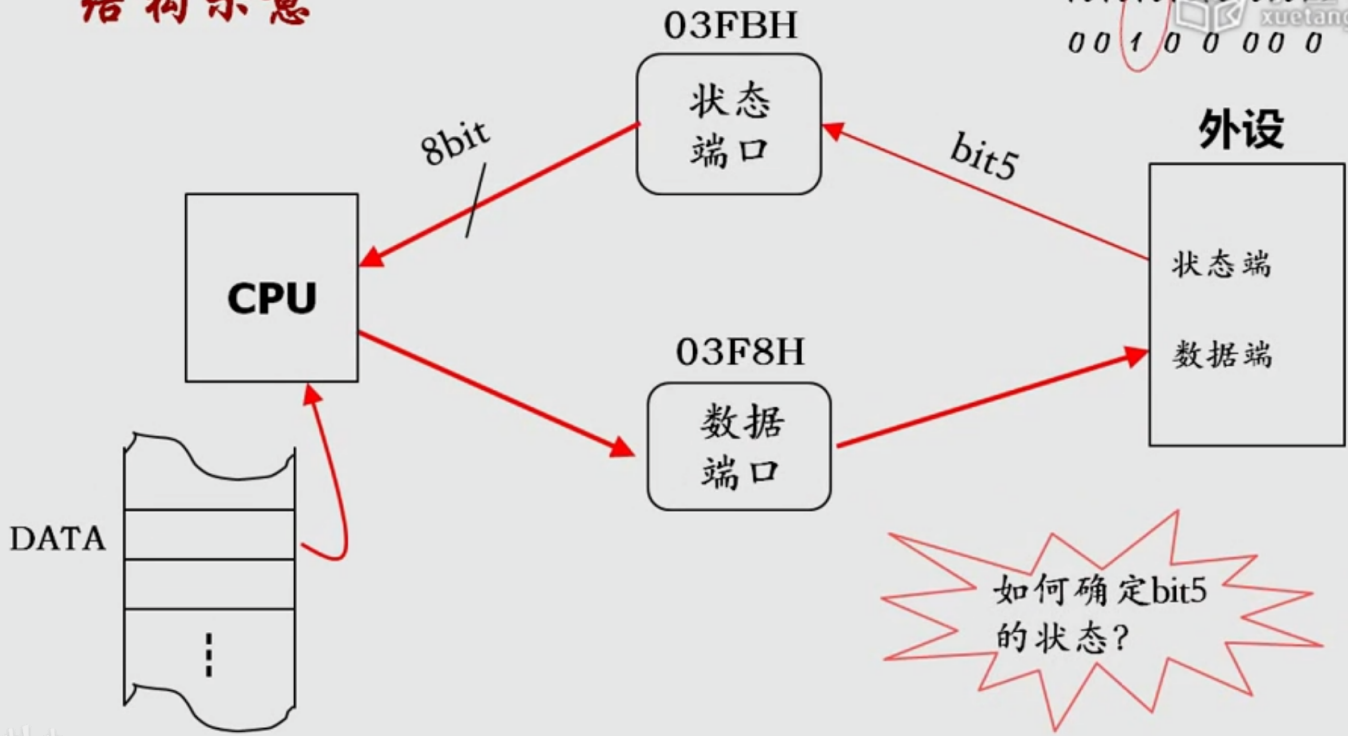


查询工作方式例

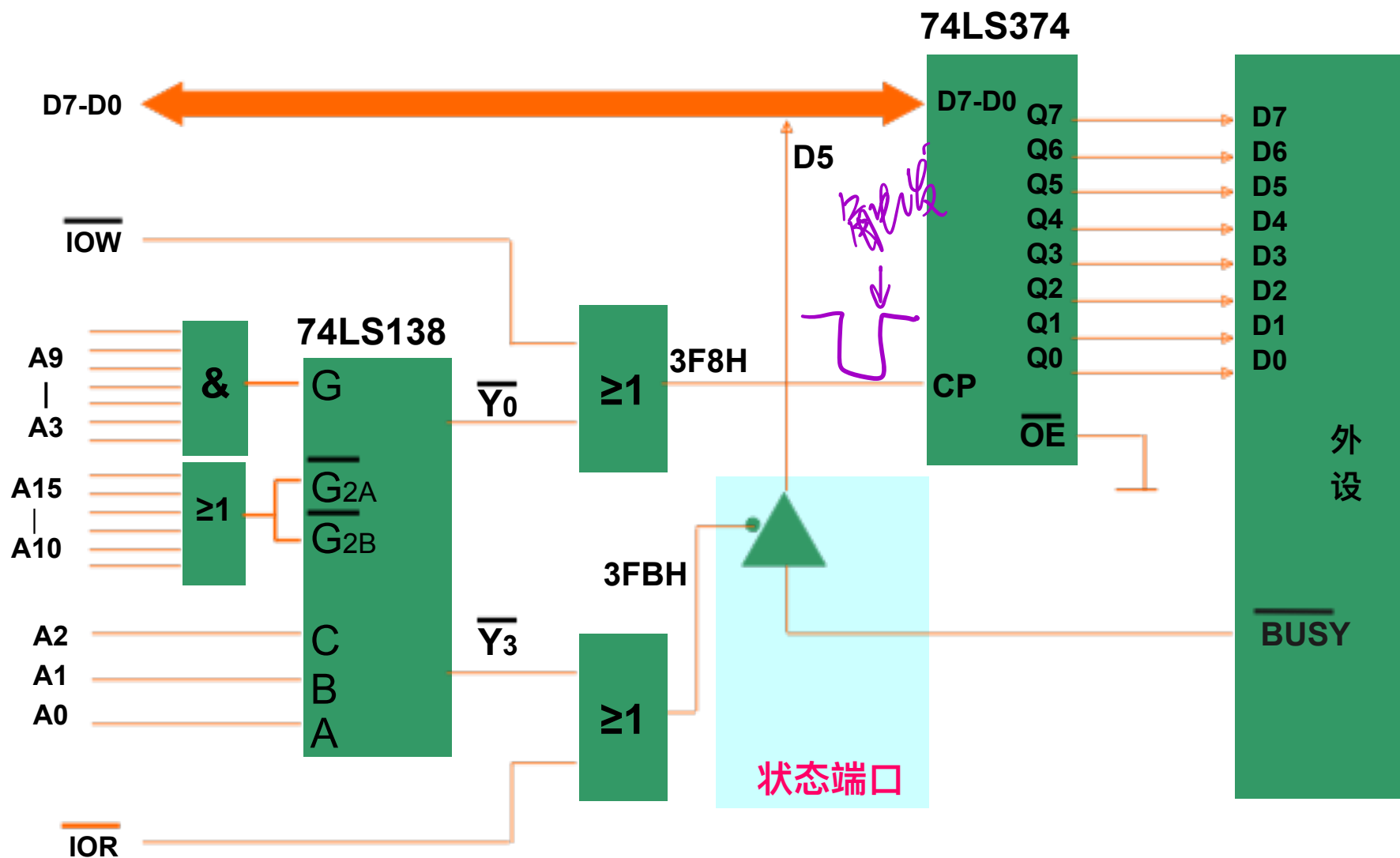
- 外设**状态端口**地址为03FBH，第5位(bit5)为状态标志（=1忙，=0准备好）
- 外设**数据端口**地址为03F8H，写入数据会使状态标志置1；外设把数据读走后又把它置0。
- 试画出其电路图，并将DATA下100B数据输出。

结构示意图

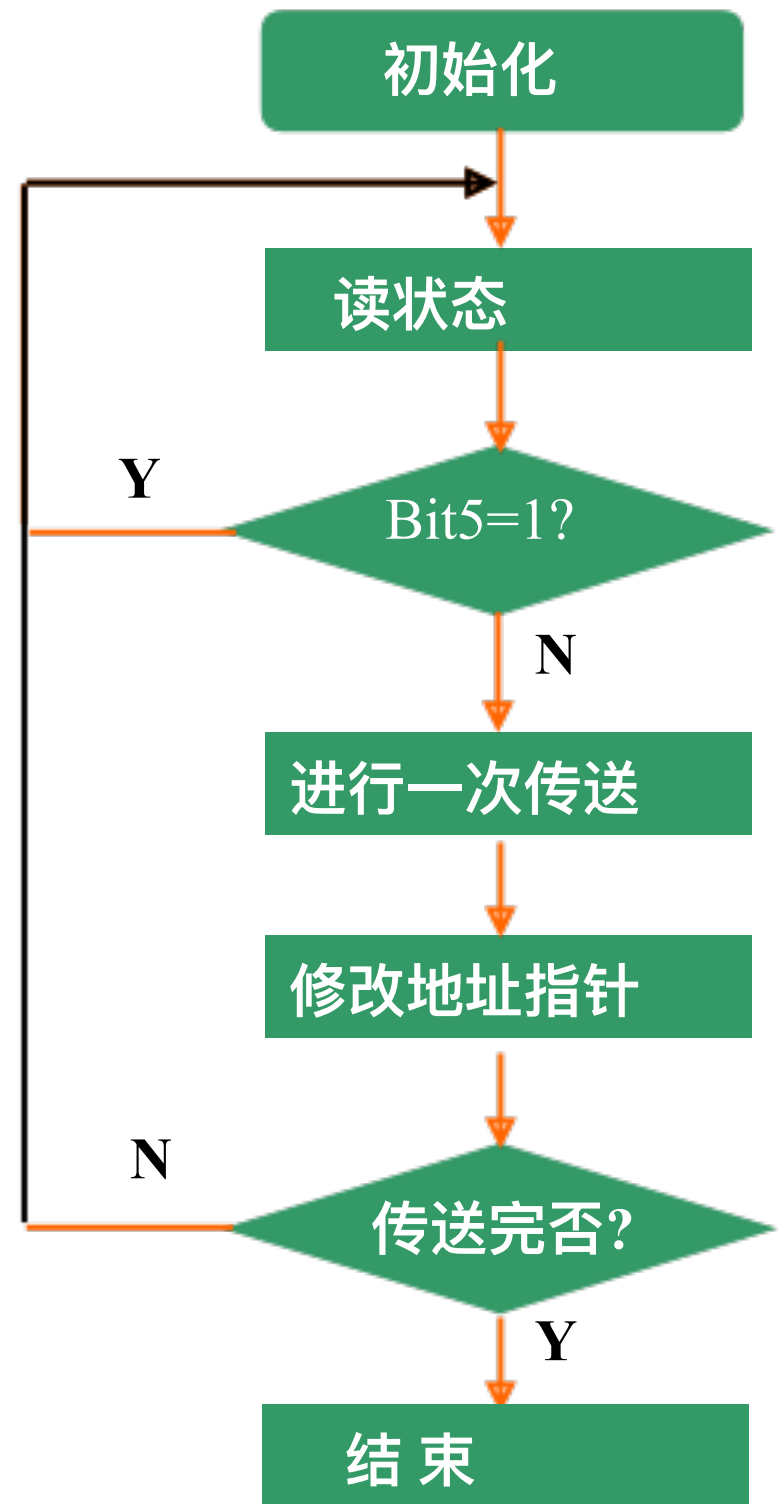
XXXXXX 在线
00100000

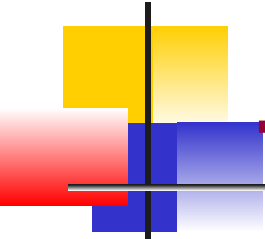


- 外设**状态端口**地址为03FBH，第5位(bit5)为状态标志 (=1忙, =0准备好)
- 外设**数据端口**地址为03F8H，写入数据会使状态标志置1；外设把数据读走后又把它置0。
- 试画出其电路图，并将DATA下100B数据输出。



```
LEA SI,DATA
MOV CX,100
AGAIN: MOV DX,03FBH
WAITT: IN AL,DX
TEST AL,20H
JNZ WAITT
MOV DX,03F8H
MOV AL, [SI]
OUT DX, AL
INC SI
LOOP AGAIN
HLT
```





三、中断控制方式

- **特点：**

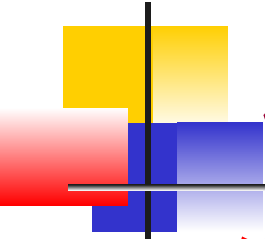
- 外设需要在需要时向CPU提出请求，CPU再去为它服务。服务结束后或在外设不需要时，CPU可执行自己的程序

- **优点：**

- CPU效率高，实时性好，速度快。

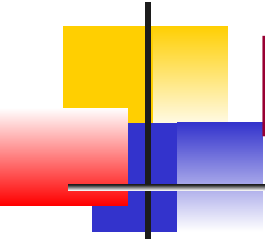
- **缺点：**

- 程序编制相对较为复杂



以上三种I/O方式的共性

- 信息的传送均需通过CPU
- 软件：
 - 外设与内存之间的数据传送是通过CPU执行程序来完成的（PIO方式）；
- 硬件：
 - I/O接口和存储器的读写控制信号、地址信号都是由CPU发出的。
- 缺点：
 - 程序的执行速度限定了传送的最大速度

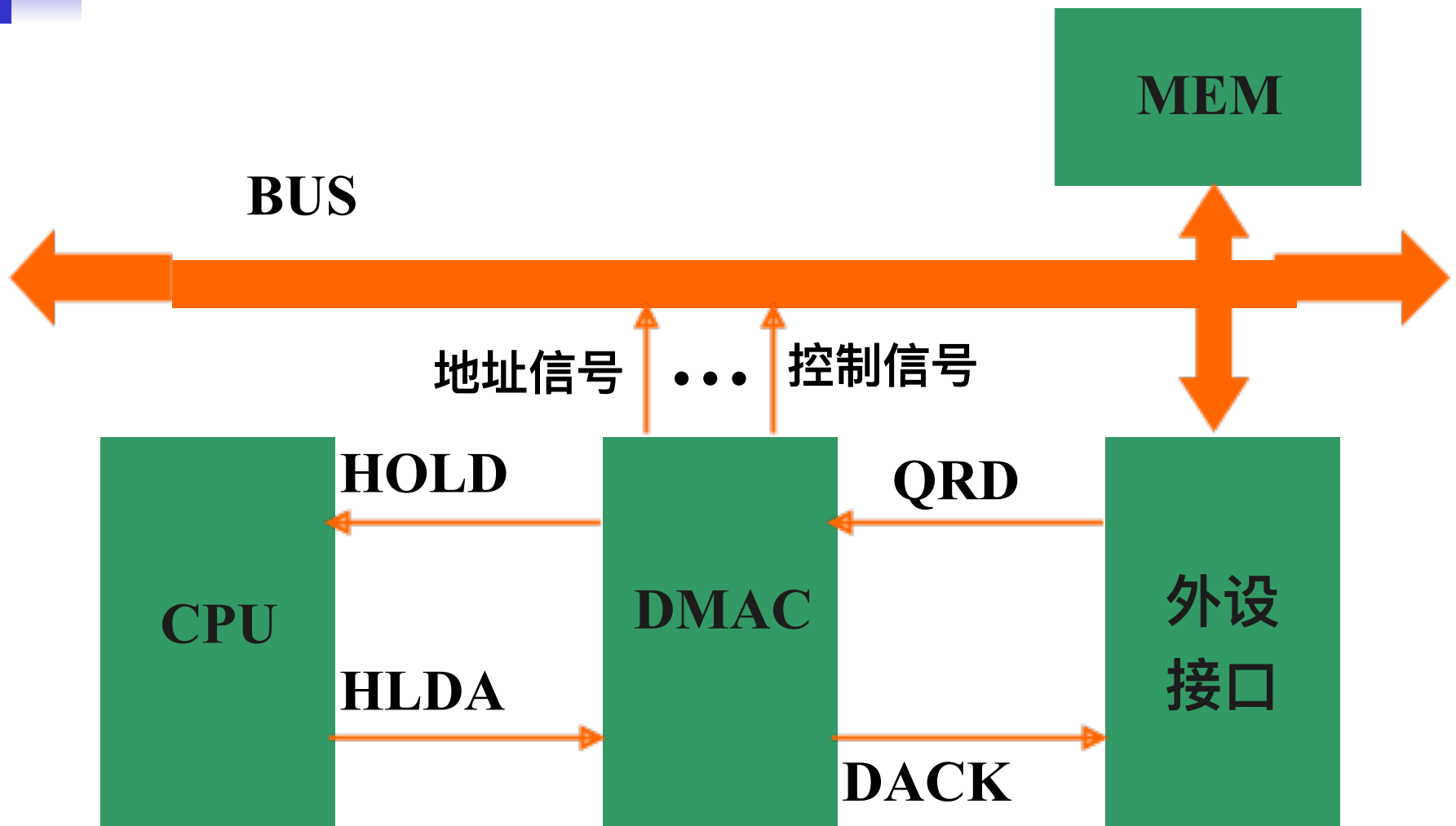


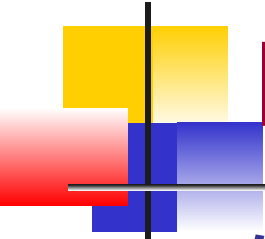
四、DMA控制方式

特点：

- 外设直接与存储器进行数据交换，CPU不再担当数据传输的中介者；
- 总线由DMA控制器（DMAC）进行控制（CPU要放弃总线控制权），内存/外设的地址和读写控制信号均由DMAC提供。

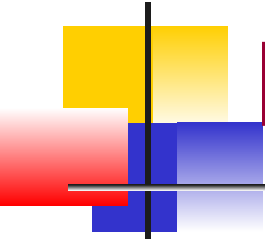
DMA控制方式





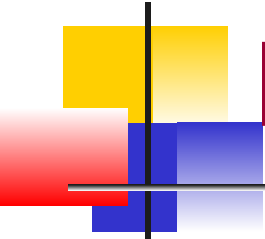
DMA控制方式的工作过程

- 外设向DMA控制器发出“DMA传送请求”信号DRQ；
- DMA控制器收到请求后，向CPU发出“总线请求”信号HOLD；
- CPU在完成当前总线周期后会立即发出HLDA信号，对HOLD信号进行响应；
- DMA控制器收到HLDA信号后，就开始控制总线，并向外设发出DMA响应信号DACK



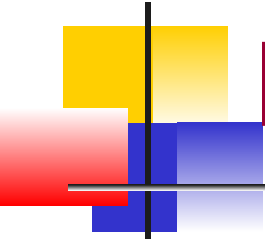
DMA控制方式的工作过程

- 例：从外设向内存传送若干字节数据
 - DMAC向I/O接口发出读信号；
 - 向地址总线上发出存储器的地址；
 - 发出存储器写信号和AEN信号；
 - 传送数据并自动修改地址和字节计数器
 - 判断是否需要重复传送操作。



DMA控制方式的工作过程

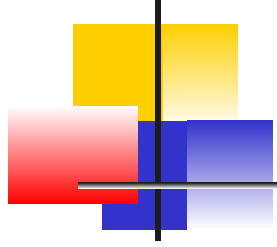
- 当规定的数据传送完后，DMA控制器就撤销发往CPU的HOLD信号。CPU检测到HOLD失效后，紧接着撤销HLDA信号，并在下一时钟周期重新开始控制总线。



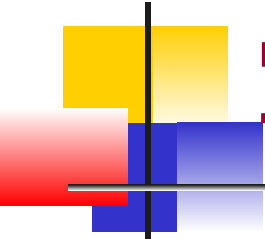
DMA控制方式

优点：

- 数据传输由DMA硬件来控制，数据直接在内存和外设之间交换，可以达到很高的传输速率。

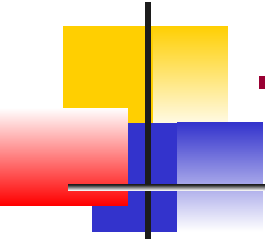


§6.4 中断技术



掌握：

- 中断的基本概念
- 中断响应的一般过程
- 中断向量表及其初始化
- 8088/8086中断系统



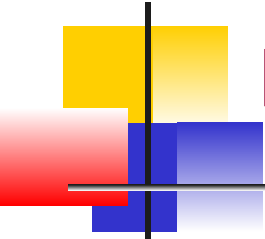
一、中断的基本概念

中断:

- CPU执行程序时，由于发生了某种随机的事件(外部或内部)，引起CPU暂时中断正在运行的程序，转去执行一段特殊的服务程序(称为中断服务程序或中断处理程序)，以处理该事件，该事件处理完后又返回被中断的程序继续执行，这一过程称为中断。

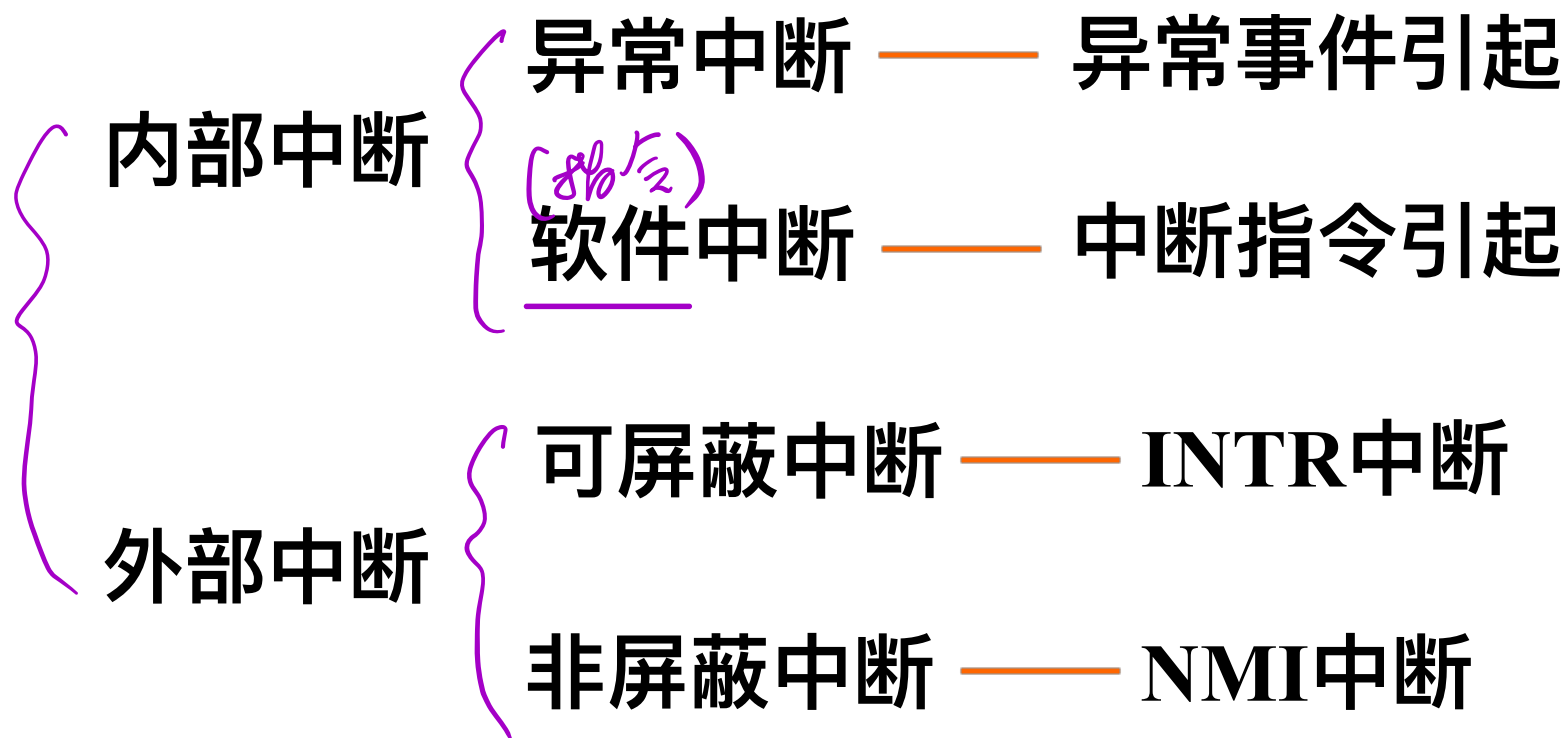
中断服务(外部)
子程序

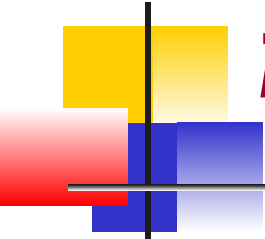




中断源

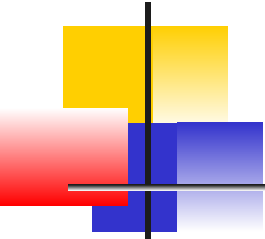
- 引起CPU中断的事件，发出中断请求的来源。





引入中断的原因

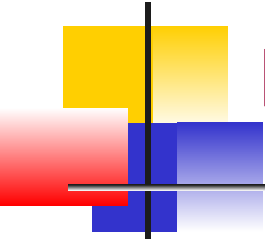
- 提高数据传输率；
- 避免了CPU不断检测外设状态的过程，提高了CPU的利用率。
- 实现对特殊事件的实时响应。



可屏蔽

二、外部中断响应的一般过程

- 中断请求
- 中断源识别及中断判优
- 中断响应
- 中断处理（服务）
- 中断返回

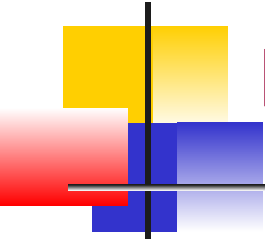


中断请求

NMI

INTR 外部可屏蔽中断请求信号

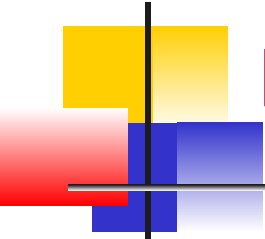
- 中断请求信号应保持到中断被处理为止；
- CPU响应中断后，中断请求信号应及时撤销。



中断源识别

- 软件查询法
- 中断矢量法
- 由中断源提供中断类型号，CPU根据类型确定中断源。

中断源识别及判优由硬件系统完成



中断判优

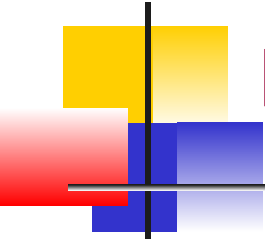
- 优先级法则

- 低优先级的中断程序允许被高优先级的中断源所中断

- 排队法则



中断嵌套



中断判优

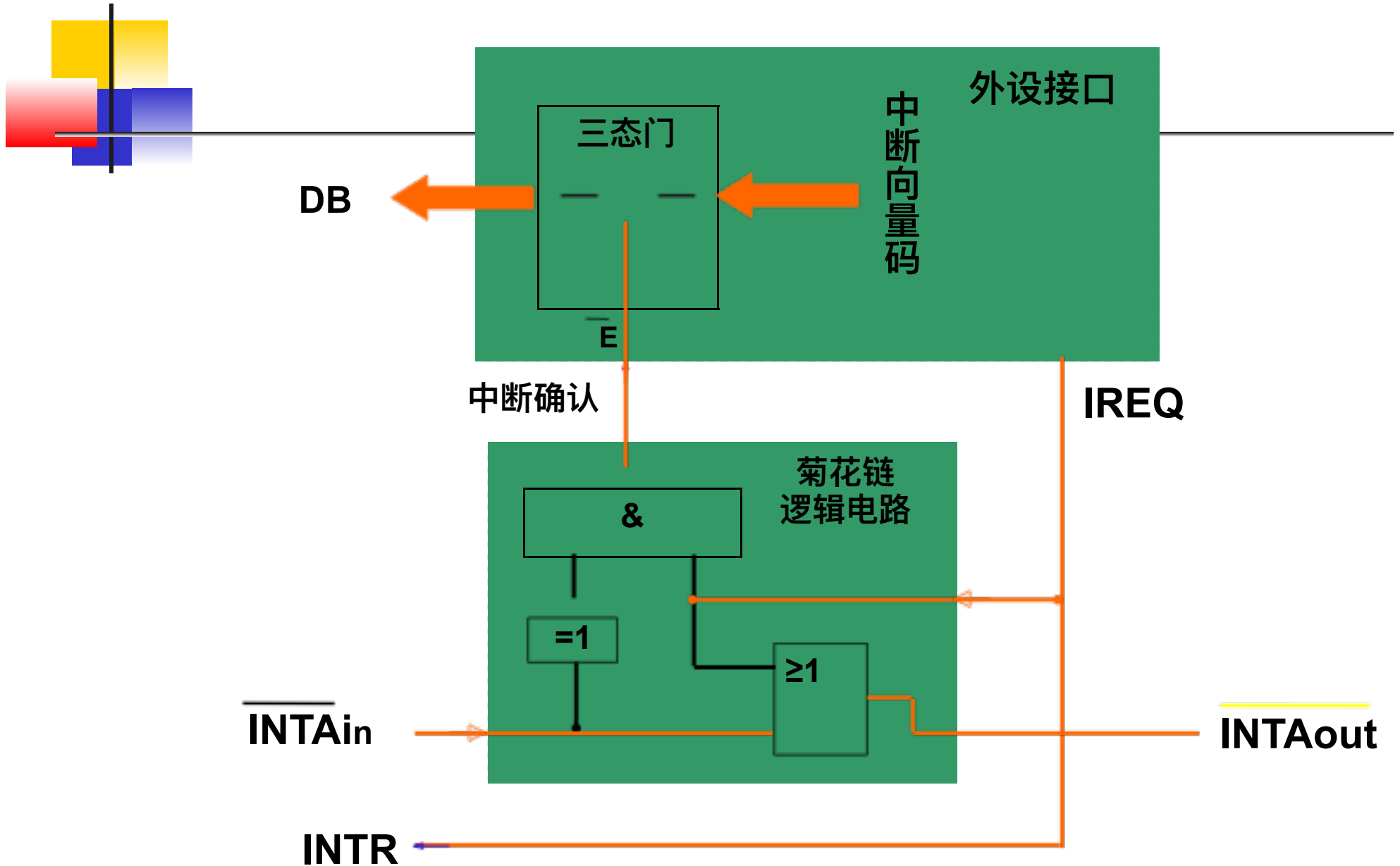
软件判优

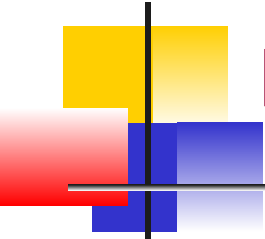
顺序查询中断请求，先查询的先服务
(即先查询的优先级别高)

硬件判优

链式判优、并行判优 (中断向量法)

菊花链逻辑电路

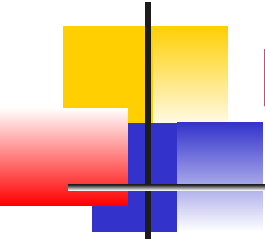




中断响应

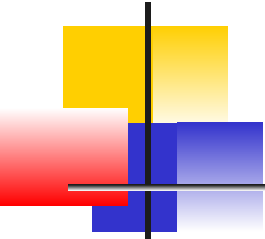
- 向中断源发出 $\overline{INTA}$ 中断响应信号；关中断；保护
硬件现场；保护断点
- 将 $\underline{FLAGS}$ 、 $\underline{CS}$ 、 $\underline{IP}$ 压入堆栈
- 获得中断服务程序入口地址

由硬件系统完成



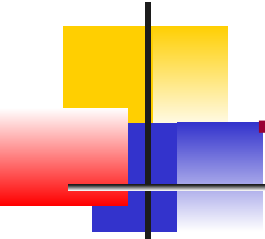
中断处理

- 执行中断服务子程序
- 中断服务子程序的特点：
 - 为“远过程”
 - 用IRET指令返回



中断服务子程序完成的工作

- 关中断，保护断点，找入口地址
- 保护软件现场（参数）
- 开中断（STI）
- 中断处理
- 关中断（CLI）
- 恢复现场 执行IRET指令，使IP、CS和FLAGS从堆栈弹出
- 中断返回



三、8088/8086中断系统

256个
中断
源

内部中断

外部中断

异常中断

软件中断

除法错中断

溢出中断

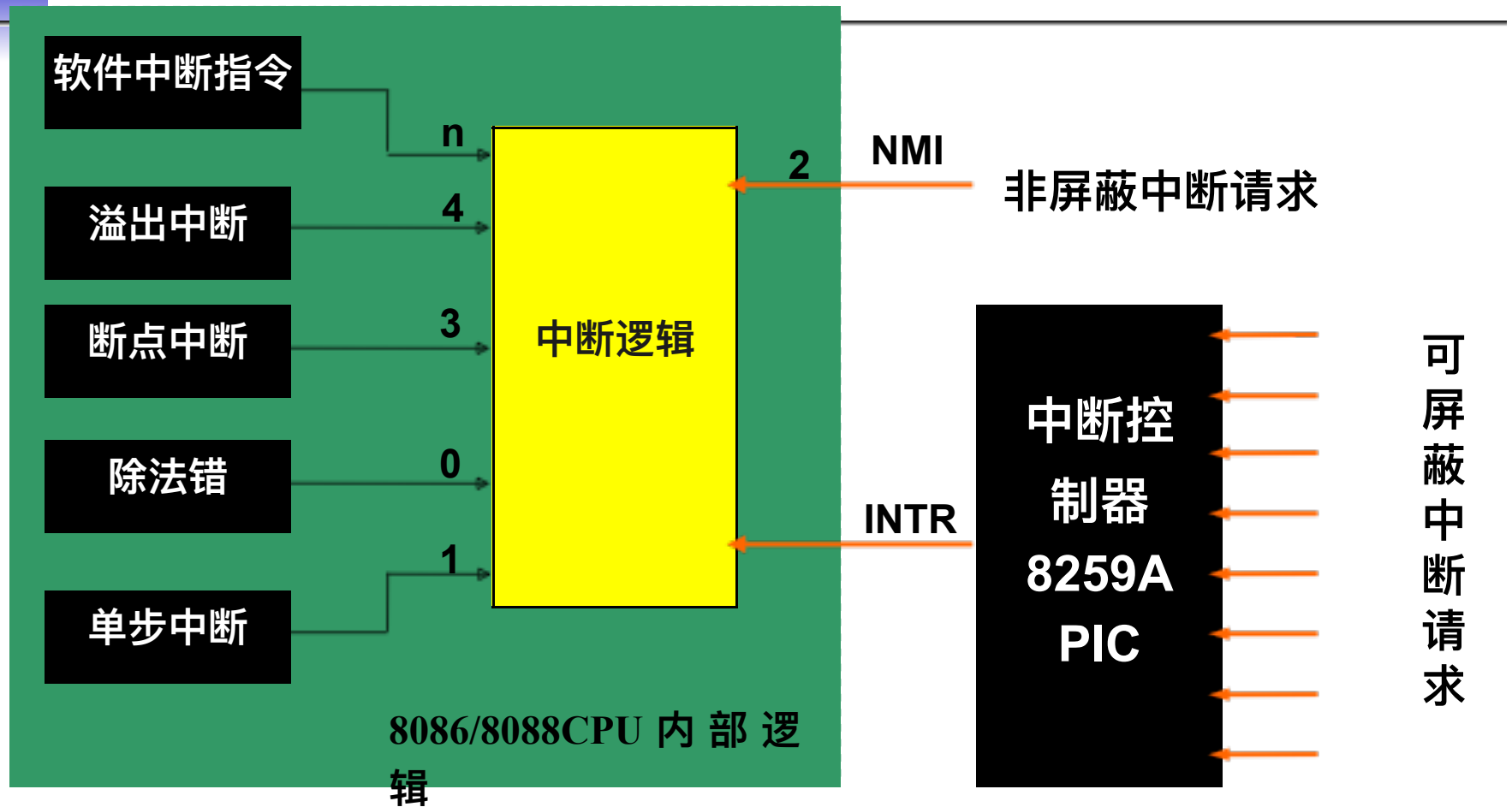
单步中断

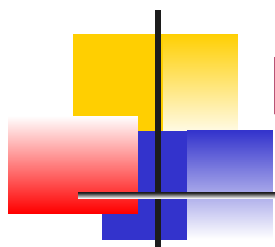
...

非屏蔽中断

可屏蔽中断

8086/8088中断源类型：

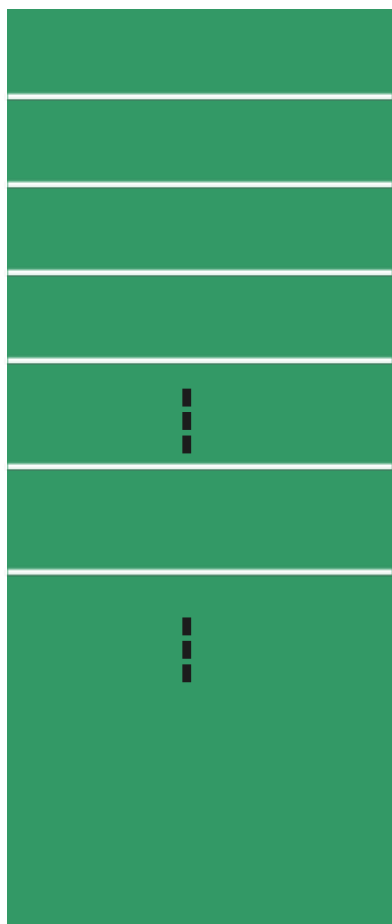




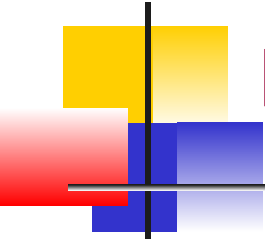
中断向量表

00000H

003FFH

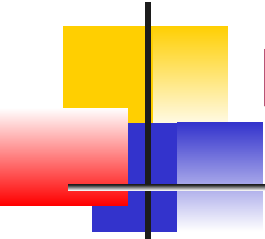


1KB



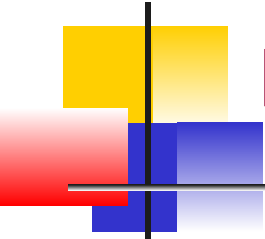
中断向量表

- 存放各类中断的中断服务程序的入口地址；
- 每个入口占用4 Bytes，低字为段内偏移，高字为段基址；
- 表的地址位于内存的00000H~003FFH，大小为1KB，共256个入口。



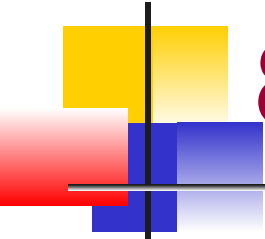
中断向量表的初始化

- 将用户自定义的中断服务程序入口地址放入向量表
- 注意点：
 - 向量表所在的段地址=0
 - 存放子程序入口的单元的偏移地址= $n \times 4$
- 例：
 - 将中断向量码为48H的服务程序入口地址放入向量表



中断向量表的初始化

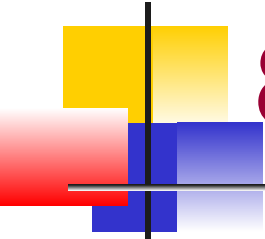
- **MOV AX, 0000H**
- **MOV DS, AX**
- **MOV SI, 0120H**
- **MOV BX, OFFSET TIMER**
- **MOV [SI], BX**
- **MOV BX, SEG TIMER**
- **MOV [SI+2], BX**



8088内部中断响应过程

特点:

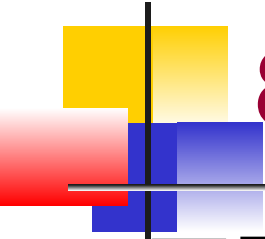
- 无INTA周期 (与其他中断的主要区别)
- 中断类型码固定或由指令给出



8088内部中断响应过程

响应过程步骤:

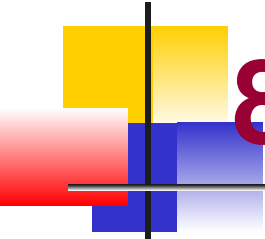
- ① PUSH FLAG
- ② LET TEMP = TF
- ③ CLEAR IF, TF
- ④ PUSH CS
- ⑤ PUSH IP
- ⑥ $(IP) = (TYPE*4+1) : (TYPE*4+0)$
- ⑦ $(CS) = (TYPE*4+3) : (TYPE*4+2)$



8088外部中断响应过程

可屏蔽中断:

- ① 第1个INTA, PIC进行优先级排队判优处理
- ② 第1个INTA, PIC把中断类型码放到DB上, 由CPU读入
- ③ PUSH FLAG
- ④ LET TEMP = TF
- ⑤ CLEAR IF, TF
- ⑥ PUSH CS
- ⑦ PUSH IP
- ⑧ $(IP) = (TYPE*4+1) : (TYPE*4+0)$
- ⑨ $(CS) = (TYPE*4+3) : (TYPE*4+2)$



8088/8086中断系统的优先级

- 优先级从高到低的顺序为：

-  内部中断
- NMI中断
-  INTR中断
- 单步中断

中断处理过程

- 当NMI、INTR、单步和除法错中断同时产生时，响应顺序为：

响应除法错中断



响应NMI中断

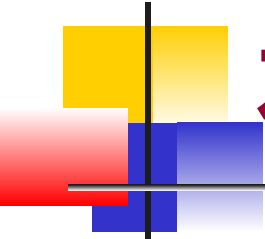


响应INTR中断



响应单步中断

级别高的中断可以中断级别低的中断服务程序



本章课外自学内容：

- 中断控制器8259